

Parte 5 - Accesso ai dati

Introduzione

Nello sviluppo di architetture a microservizi, la gestione dello stato e la persistenza dei dati rappresentano una delle sfide ingegneristiche più complesse. Ogni microservizio dovrebbe idealmente possedere e gestire il proprio database per garantire l'isolamento e l'indipendenza (pattern *Database-per-Service*).

Il Modello Relazione e i Vincoli di Integrità

Prima di scrivere codice, è fondamentale comprendere come i dati vengono strutturati. Il **modello relazionale** si basa su un solido fondamento matematico, pur presentandosi all'utente in una forma estremamente intuitiva. Esso si fonda su due concetti chiave: la **tabella** (intuitiva) e la **relazione** (formale e matematica).

La teoria degli insiemi alla base dei dati

Dal punto di vista formale, una relazione matematica nasce dal concetto di prodotto cartesiano. Dati n insiemi $D_1, D_2, \dots, D_n$ (chiamati *domini*), il prodotto cartesiano $D_1 \times D_2 \times \dots \times D_n$ è l'insieme di tutte le possibili n -uple (o tuple) formattate in questo modo: $\{(d_1, \dots, d_n) : d_i \in D_i \forall i = 1, \dots, n\}$.

Una "relazione" nel nostro database non è altro che un **sottoinsieme** matematico di questo vasto prodotto cartesiano, che contiene solo le n -uple che hanno un significato logico per la nostra applicazione.

Nella pratica di un DataBase Management System (DBMS), questa struttura si traduce in una **tabella**, composta da righe e colonne:

- **Riga (n-upla o tupla):** Rappresenta un singolo oggetto logico in cui i valori sono correlati. Essendo le tuple viste come sequenze non ordinate di dati, utilizziamo le intestazioni per capirne il significato.
- **Colonna (attributo):** Definisce il "ruolo" di un dato all'interno della riga, permettendoci di distinguere i valori.

I dati memorizzati in queste colonne devono appartenere a tipi specifici, come caratteri (char, varchar), numerici esatti o approssimati (integer, float), istanti temporali (date, timestamp) o formati binari.

I Vincoli di Integrità

In un sistema reale, non tutte le combinazioni di dati sono lecite. Per evitare di inserire dati errati ("garbage in, garbage out"), i DBMS implementano regole booleane (vere o false) chiamate **vincoli di integrità**. Se una tupla non rispetta il vincolo (valore *false*), il database rifiuta l'operazione. Si dividono in:

1. **Vincoli di tupla:** Operano sulla singola riga. Ad esempio, in una tabella *Esami*, un vincolo può stabilire che il "Voto" debba essere compreso tra 18 e 30, o che la "Lode" sia valorizzata a "sì" solo se il voto è 30.
2. **Vincoli di chiave (Primary Key):** Un insieme di colonne che garantisce che non esistano due righe identiche in una tabella. Questo identifica univocamente ogni record e permette di stabilire corrispondenze tra tabelle.
3. **Vincoli di integrità referenziale (Foreign Key):** Regolano le relazioni *tra* tabelle. Se la tabella T_1 ha una colonna C che fa riferimento alla tabella T_2 , i valori in C devono esistere obbligatoriamente come chiavi primarie in T_2 .

Accesso a Basso Livello con ADO.NET

Nel mondo Microsoft .NET, prima dell'avvento dei framework astratti, l'accesso ai dati era (ed è tuttora sotto il cofano) governato da **ADO.NET**. ADO.NET fornisce un insieme di strumenti per interagire con i database

attraverso due filosofie principali:

1. Architettura Connessa: il Data Provider

È l'approccio diretto e performante. I componenti essenziali sono:

- **Connection:** stabilisce un canale di comunicazione fisico con il DB usando una stringa di connessione.
- **Command:** esegue istruzioni SQL dirette (SELECT, UPDATE) o Stored Procedure.
- **DataReader:** fornisce un flusso di lettura (forward-only e in sola lettura) estremamente veloce dei dati restituiti da una query.

```
// Esempio pratico di lettura dati con ADO.NET Data Provider
using Microsoft.Data.SqlClient;
string connectionString = "Server=localhost, 2433; Database=MyDB; User Id=sa; Password=p4ssw0rD; Encrypt=False";
using SqlConnection connection = new (connectionString);
string cmdText = "SELECT Matricola, Cognome FROM Studenti;";
using SqlCommand command = new (cmdText, connection);

await connection.OpenAsync();
using SqlDataReader reader = await command.ExecuteReaderAsync();
while (await reader.ReadAsync())
{
    Console.WriteLine("{0}\t{1}", reader.GetInt32(0), reader.GetString(1));
}
```

2. Architettura Disconnessa: il DataSet

Il **DataSet** è un contenitore in memoria, totalmente scollegato dal database di origine. Ricrea al suo interno una "mini-replica" del database, contenente collezioni di `DataTable`, `DataRow`, `DataColumn` e persino replicando i vincoli di chiave e le relazioni (`DataRelationCollection`). Per riempirlo si utilizza il **DataAdapter**, che fa da ponte temporaneo tra il DB fisico e il DataSet in memoria.

```
using Microsoft.Data.SqlClient;
string connectionString = "Server=localhost,2433;Database=MyDB;User Id=sa;Password= p4ssw0rD;Encrypt=False";
using SqlConnection connection = new (connectionString);
string queryString = "SELECT Matricola, Cognome FROM Studenti";
using SqlDataAdapter adapter = new (queryString, connection);
DataSet customers = new ();
adapter.Fill(customers, "Studenti");
```

L'Infrastruttura di Sviluppo (SQL Server su Docker)

Nello sviluppo di microservizi, installare pesanti motori di database in locale è un anti-pattern. La soluzione moderna è la containerizzazione. Tramite Docker, possiamo avviare un'istanza di SQL Server e un'interfaccia di amministrazione in pochi secondi, garantendo che ogni sviluppatore del team abbia l'esatto medesimo ambiente.

Il file `docker-compose.yml` fornito negli appunti illustra questo concetto associando il servizio `mssql-server` (con l'accettazione della licenza, password sicura e mappatura della porta `2433` sull'host verso la `1433` del container) a un volume persistente (`/var/opt/mssql`), e affiancandogli `adminer` come client web per visualizzare comodamente i dati sulla porta `2431`.

```
version: '3.4'
services:
  mssql-server:
    image: "mcr.microsoft.com/mssql/server:2019-latest"
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_PID: Developer
      MSSQL_SA_PASSWORD: p4ssw0rd
    ports:
      - 2433:1433
    volumes:
      - "mssql-server:/var/opt/mssql"
  adminer:
    image: adminer:4.8.1
    ports:
      - 2431:8080
    environment:
      TZ: Europe/Rome
      ADMINER_DEFAULT_SERVER: mssql-server
    volumes:
      mssql-server:
```

Il Paradigma ORM (Object-Relational Mapping)

Se ADO.NET è performante, perché utilizziamo altri strumenti? Il problema risiede nel cosiddetto "Impedance Mismatch": i database relazionali parlano il linguaggio delle tabelle, righe e colonne; C# (come Java o Python) parla il linguaggio della programmazione orientata agli oggetti (OO), fatto di classi, ereditarietà e polimorfismo.

L'**ORM (Object Relational Mapping)** è una tecnica software che risolve questo conflitto: mappa letteralmente le tabelle in classi, i record in istanze (oggetti) e le colonne in proprietà.

Vantaggi dell'ORM:

- **Portabilità:** Il codice C# rimane lo stesso anche se cambiamo il DBMS sottostante (es. da SQL Server a PostgreSQL).
- **Livello di astrazione:** Permette di scrivere codice ad alto livello, isolando la logica di accesso al DB e riducendo drasticamente il codice *boilerplate* (operazioni CRUD risolte in poche righe).
- **Sicurezza:** Offre protezione intrinseca da attacchi di SQL Injection elaborando in sicurezza i parametri.

Svantaggi dell'ORM:

- Può introdurre una **riduzione delle prestazioni** per query estremamente complesse, dove la scrittura in SQL nativo (bypassando l'ORM) risulta l'unica via praticabile.
- Presenta una **curva di apprendimento** iniziale non trascurabile.

Entity Framework Core (EF Core)

Entity Framework Core è il framework ORM di punta, open-source e multiplatforma di casa Microsoft, evoluzione del vecchio Entity Framework. Tramite una libreria di provider, si aggancia a DBMS diversi (SQL Server, MongoDB, Oracle, ecc.) consentendo di manipolare i dati attraverso oggetti .NET fortemente tipizzati.

Il Modello di EF

Il modello definisce la forma del nostro database nel codice. Sostanzialmente, è un insieme di classi C#. Possiamo operare in due modi:

1. **Database First:** Generiamo le classi a partire da un database esistente.
2. **Code First (Raccomandato):** Scriviamo noi le classi C# (i *Model*) e usiamo il sistema di "Migrazioni" per fare in modo che EF Core generi e aggiorni automaticamente lo schema del database per noi.

```
public class Studenti
{
    public int Matricola { get; set; }
    public required string Cognome { get; set; }
    public required string Nome { get; set; }
    public DateTime DataDiNascita { get; set; }
    public List<Esami> ListaEsami { get; set; }
}
```

Il DbContext

Il **DbContext** è il direttore d'orchestra. Una sua istanza rappresenta una sessione di lavoro singola e di breve durata con il database.

Esso contiene proprietà di tipo **DbSet<TEntity>**, che agiscono come collezioni in memoria delle tabelle. Ogni query eseguita su un **DbSet** viene automaticamente tradotta dal provider in linguaggio SQL nativo (es. T-SQL) ed eseguita sul DBMS.

Ciclo di vita del DbContext:

1. Creazione dell'istanza.
2. *Tracking* (tracciamento): quando leggiamo, inseriamo o modifichiamo entità, il context le "osserva" tenendo traccia delle variazioni in memoria.
3. Chiamata a **SaveChanges()**: tutte le variazioni tracciate vengono tradotte in istruzioni SQL **INSERT**, **UPDATE** o **DELETE** ed eseguite sul database in una singola transazione.
4. Eliminazione (Dispose): Essendo un oggetto che implementa **IDisposable** e consuma risorse di rete/memoria, deve essere distrutto a fine utilizzo. Nelle architetture moderne, la *Dependency Injection* lo gestisce iniettandolo come servizio di tipo *Scoped*.

Attenzione per ambienti multi-thread: Il DbContext **non è thread-safe**. Non deve mai essere condiviso tra thread paralleli contemporaneamente.

```
public class UniprDbContext : DbContext
{
    public UniprDbContext(DbContextOptions<UniprDbContext> options) : base(options) { }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```

{
    // configurazione del modello tramite API fluent
}
public DbSet<Studenti> Studenti { get; set; }
public DbSet<Corsi> Corsi { get; set; }
public DbSet<Esami> Esami { get; set; }
}

```

Configurazione del Modello e Relazioni

Come fa EF Core a capire che la classe `Studenti` deve mappare una tabella chiamata "AnagraficaStudenti" o qual è la chiave primaria? Usa un sistema di mapping dei metadati, applicabile in due modi:

1. **Data Annotations:** Attributi inseriti direttamente sopra le proprietà delle classi entità (es. `[Table("Studenti")]`, `[Key]`). È rapido, ma "sporca" le classi di dominio con dettagli infrastrutturali.

```

[Table("Studenti")]
[PrimaryKey("Matricola")]
public class Studenti
{
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Matricola { get; set; }
    public required string Cognome { get; set; }
    public required string Nome { get; set; }
    public DateTime DataDiNascita { get; set; }
    public List<Esami> ListaEsami { get; set; } = new List<Esami>();
}

```

2. **Fluent API:** La configurazione avviene centralmente sovrascrivendo il metodo `OnModelCreating` dentro il `DbContext`. Permette di mappare i nomi delle colonne, precisioni, vincoli di nullabilità e chiavi primarie senza toccare le classi di base (es. `modelBuilder.Entity<Corsi>().HasKey(c => c.Codice);`).

```

public class UniprDbContext : DbContext
{
    public UniprDbContext(DbContextOptions<UniprDbContext> options) : base(options)
    { }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // configurazione del modello tramite API fluent
        modelBuilder.Entity<Studenti>().ToTable("Studenti");
        modelBuilder.Entity<Studenti>().HasKey(s => s.Matricola);
        modelBuilder.Entity<Corsi>().HasKey(c => c.Codice);
        modelBuilder.Entity<Esami>().HasKey(e => new { e.CorsiId, e.StudentiId });
    }
    public DbSet<Studenti> ListaStudenti { get; set; }
    public DbSet<Corsi> Corsi { get; set; }
    public DbSet<Esami> Esami { get; set; }
}

```

Un aspetto vitale sono le **Relazioni**. Se uno studente può fare più esami, tra la classe `Studenti` ed `Esami` c'è una relazione "1 a Molti". In EF Core questo si rappresenta con le **Navigation Properties**: la classe `Studenti` avrà una proprietà `List<Esami>`, mentre `Esami` avrà un riferimento singolo all'oggetto `Studenti` e una proprietà per la chiave esterna (`StudentiId`). Se rispettiamo le convenzioni di nomenclatura, EF Core deduce le relazioni automaticamente; altrimenti, possiamo specificarle esplicitamente tramite la Fluent API (`.HasMany(...).WithOne(...)`).

Esecuzione delle Query (LINQ)

Invece di scrivere query SQL come facevamo in ADO.NET, EF Core ci permette di usare **LINQ (Language-Integrated Query)**. LINQ analizza il nostro codice C#, genera una struttura logica della query (Expression Tree) e la passa al Provider del database. Quest'ultimo traduce elegantemente questa espressione nel dialetto SQL appropriato.

```
var contextOptions = new DbContextOptionsBuilder<UniprDbContext>()
    .UseSqlServer(
        "Server=localhost,2433;Database=UNIPR;User Id=sa;Password=p4ssw0rD;Encrypt=False")
    .Options;
using UniprDbContext context = new UniprDbContext(contextOptions);
var studenti = await context.Studenti.ToListAsync();
```